CS-2710-1 Spring 2026

Computer Organization and Systems (COS)

Lecture #15

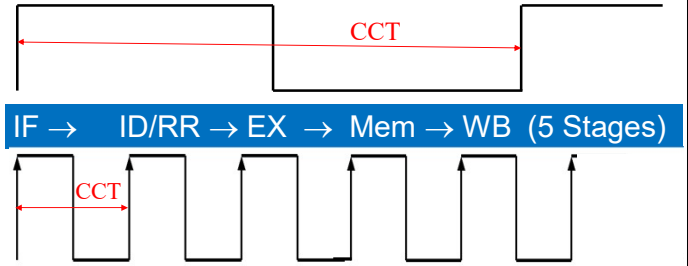
01 April 2026

Ashoka University

BBB

Processor Design

Single-Cycle Processor (one instruction/clock cycle)

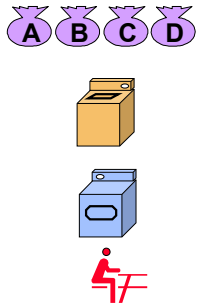


Multi-Cycle Processor (one stage/clock cycle)

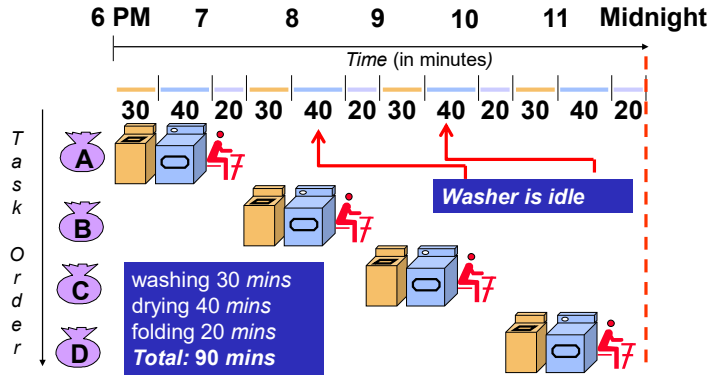
- CPU time = IC * CPI * CCT
  - Single-cycle machine needs large CCT ⇒ CPU-time ↑
  - Multi-cycle processors support pipelining
    ⇒ CCT/CPI ↓ ⇒ CPU-time ↓ ⇒ performance ↑

What is Pipelining

- Laundry Room
- Four students A, B, C, D each has one load of clothes to wash, dry, and fold
- Washer takes 30 minutes
- Dryer takes 40 minutes
- Folder takes 20 minutes



What Is Pipelining: Laundry Example

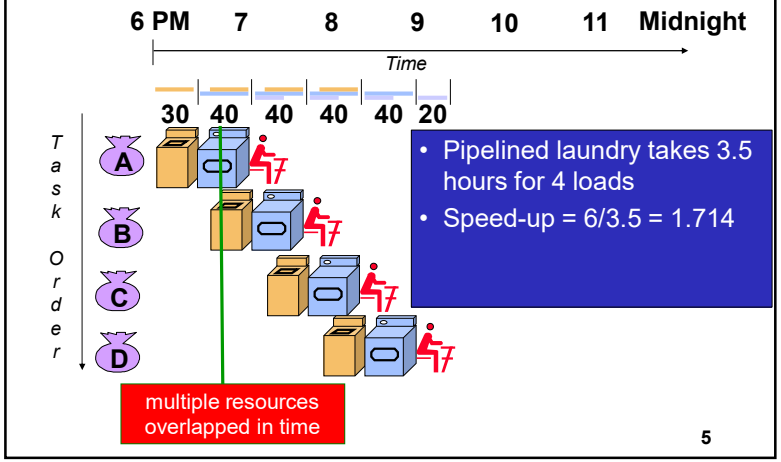


Sequential laundry takes 6 hours for four users; Work finishes at 00:00
If they learned pipelining, how long would laundry take?

Only one resource is busy at a time

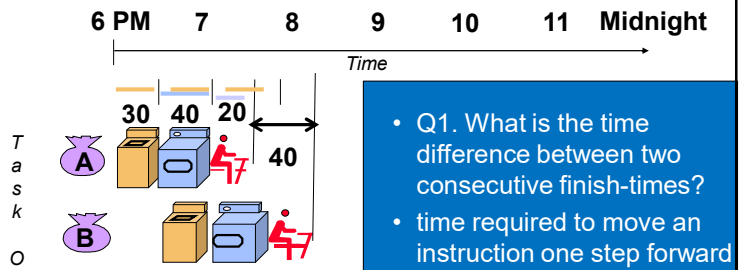
Basic Idea of Pipelining

Minimize idle-time for resources



- Pipelined laundry takes 3.5 hours for 4 loads
- Speed-up = 6/3.5 = 1.714

Impact of Pipeline Stages

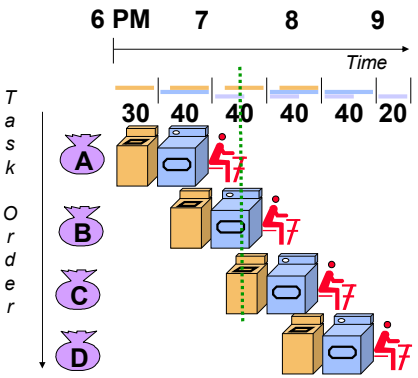


- Q1. What is the time difference between two consecutive finish-times?
- time required to move an instruction one step forward
- → longest time (slowest) among pipe-stages

Q2. What is the maximum speed-up compared to non-pipelined approach??
→ limited by the number of pipe-stages

- Speed-up when $n \to \infty$
  $\approx 90/40 = 2.25$
  < 3 (number of pipe-stages)

Pipelining Lessons



- Pipelining doesn't help **latency** of single task, it helps **throughput** of entire workload
- Pipeline rate limited by **slowest** pipeline stage
- **Multiple** tasks operating simultaneously
- Potential speed-up ≤ **Number pipe stages (pipe-depth)**
- Unbalanced lengths of pipe stages reduces speed-up
- Time to "**fill**" pipeline and time to "**drain**" it reduces speedup
- **Stall for Dependences**

Learned So Far ….

- Pipelining is widely used in modern processors.
- Pipelining improves system performance in terms of throughput.
- Pipelined organization requires sophisticated compilation techniques.

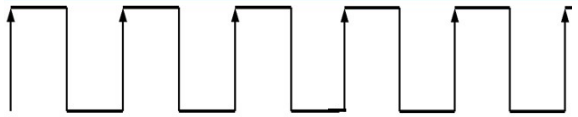
Making Program Execution Faster

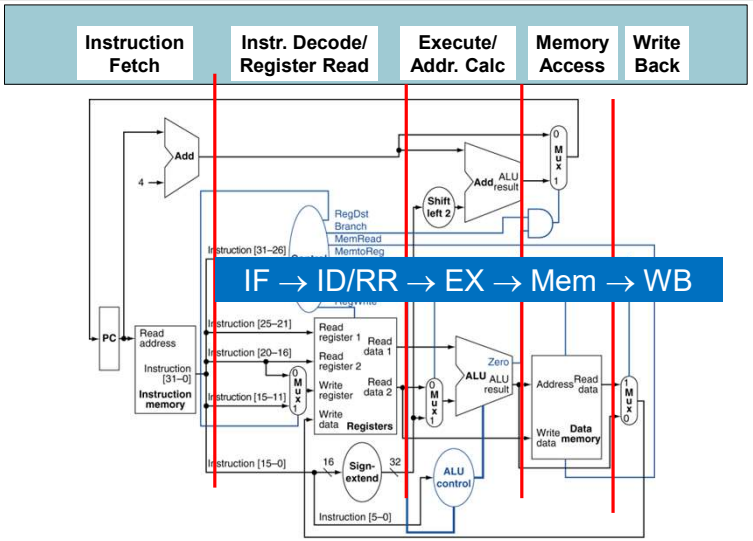
- Arrange the hardware so that more than one operation can be overlapped at the same time.
- The number of operations performed per second is increased even though the elapsed time needed to perform any one operation is not changed.

MIPS Pipeline

- Five stages, one step per stage
  1. IF: Instruction fetch from memory
  2. ID/RR: Instruction decode & register read
  3. EX: Execute operation or calculate address
  4. MEM: Access memory operand
  5. WB: Write result back to register

IF → ID/RR →EX → Mem → WB





A pipeline diagram

Clock cycle (time)

		1	2	3	4	5	6	7	8	9
lw	$t0, 4($sp)	IF	ID	EX	MEM	WB				
sub	$v0, $a0, $a1		IF	ID	EX	MEM	WB			
and	$t1, $t2, $t3			IF	ID	EX	MEM	WB		
or	$s0, $s1, $s2				IF	ID	EX	MEM	WB	
add	$sp, $sp, -4					IF	ID	EX	MEM	WB

- A pipeline diagram shows the execution of a series of instructions.
  - The instruction sequence is shown vertically, from top to bottom.
  - Clock cycles are shown horizontally, from left to right.
  - Each instruction is divided into its component stages.
- Overlapping of instructions: five instructions are active in the fifth cycle:
- Why balanced stages??

- logistics;
- speed-up is determined by the time needed by the longest stage, anyway;

Pipeline terminology

Clock cycle

		1	2	3	4	5	6	7	8	9
lw	$t0, 4($sp)	IF	ID	EX	MEM	WB				
sub	$v0, $a0, $a1		IF	ID	EX	MEM	WB			
and	$t1, $t2, $t3			IF	ID	EX	MEM	WB		
or	$s0, $s1, $s2				IF	ID	EX	MEM	WB	
add	$sp, $sp, -4					IF	ID	EX	MEM	WB

filling full emptying

- The pipeline depth is the number of stages—in this case, five.
- In cycle 5, the pipeline is full. Five instructions are being executed simultaneously, so all hardware units are in use => speed-up $\leq$ 5

Pipeline Datapath: Resource Requirements

Clock cycle

		1	2	3	4	5	6	7	8	9
lw	$t0, 4($sp)	IF	ID	EX	MEM	WB				
lw	$t1, 8($sp)		IF	ID	EX	MEM	WB			
lw	$t2, 12($sp)			IF	ID	EX	MEM	WB		
lw	$t3, 16($sp)				IF	ID	EX	MEM	WB	
lw	$t4, 20($sp)					IF	ID	EX	MEM	WB

- We need to perform several operations in the same cycle.
  - Increment the PC and add registers at the same time.
  - Fetch one instruction while another one reads or writes data.
- What does this mean for our hardware?

Pipelining Multiple Instruction-Types

- R-type instructions only require 4 stages: IF, ID, EX, and WB
  - We don't need the MEM stage
- What happens if we try to pipeline **loads** with **R-type** instructions?

Clock cycle

		1	2	3	4	5	6	7	8	9
add	$sp, $sp, -4	IF	ID	EX	WB					
sub	$v0, $a0, $a1		IF	ID	EX	WB				
lw	$t0, 4($sp)			IF	ID	EX	MEM	WB		
or	$s0, $s1, $s2				IF	ID	EX	WB		
lw	$t1, 8($sp)					IF	ID	EX	MEM	WB

Solution??

A solution: Insert NOP (No-Operation) stages

- Enforce uniformity
  - Ensure that all instructions take 5 cycles/stages
  - Make them have the same stages, in the same order
    - Some stages will do nothing for some instructions

R-type	IF	ID	EX	NOP	WB

Clock cycle

		1	2	3	4	5	6	7	8	9
add	$sp, $sp, -4	IF	ID	EX	NOP	WB				
sub	$v0, $a0, $a1		IF	ID	EX	NOP	WB			
lw	$t0, 4($sp)			IF	ID	EX	MEM	WB		
or	$s0, $s1, $s2				IF	ID	EX	NOP	WB	
lw	$t1, 8($sp)					IF	ID	EX	MEM	WB

Mem: NOP

- Stores and Branches have NOP stages, too…

store	IF	ID	EX	MEM	NOP
branch	IF	ID	EX	NOP	NOP

WB: NOP
Mem: NOP,
WB: NOP

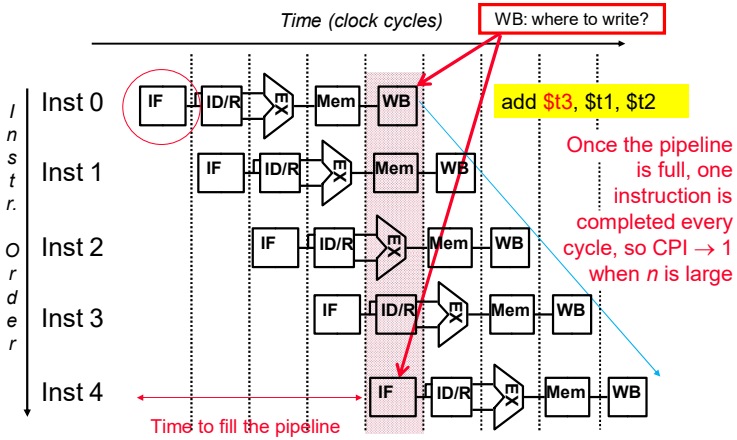
Learned So Far ….

- Pipelining attempts to maximize instruction throughput by overlapping, in time, the execution of multiple instructions.
- All instructions must pass through the same number of stages (required plus NOPs)
- Pipelining offers speed-up that is bounded by the number of pipe-stages
  - In the best case, one instruction finishes on every cycle, and the speed-up is equal to the pipeline depth.
  - Each stage needs is own functional units

Pipeline Speed-up

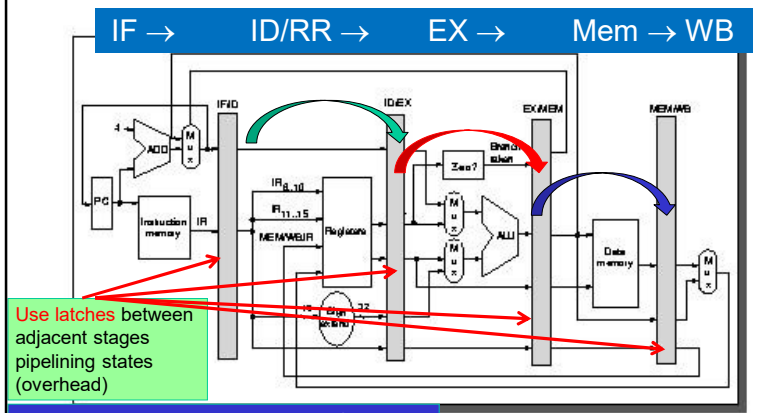
- If all stages are balanced
  - i.e., all take the same time
  - Speed-up $\leq k$ (pipe-depth)
- If not balanced, speed-up is less
- However, life is not that simple; actual speed-up might be further affected due to:
- -- pipeline overhead
- -- pipe hazards

Pipeline issues



Information regarding destination-register-ID for Inst-0 is no longer available, because Inst-4 has already entered into the pipeline

Solution??



Use latches between adjacent stages pipelining states (overhead)

Latch-contents should be propagated forward as the pipeline advances

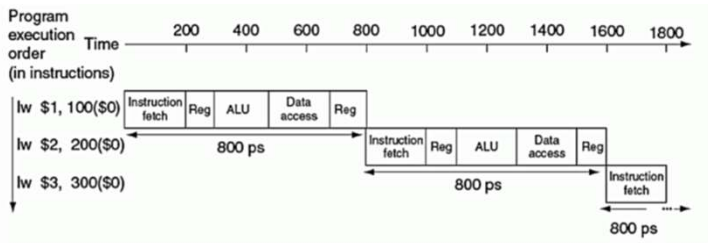
Cons: Adds pipeline overhead; slows down the pipeline

Pipelining: Executing a Program Faster

- Pipelining involves executing an instruction over multiple clock cycles
- Most importantly, it enables executing multiple instructions per cycle
- In other words, multiple instructions are overlapped in time, thus enabling speed-up
- However, in every cycle, each resource is only used by at most one instruction
- CPI reduces (ideally, CPI becomes equal to 1 in the absence of hazards)

Pipelining: Executing a Program Faster

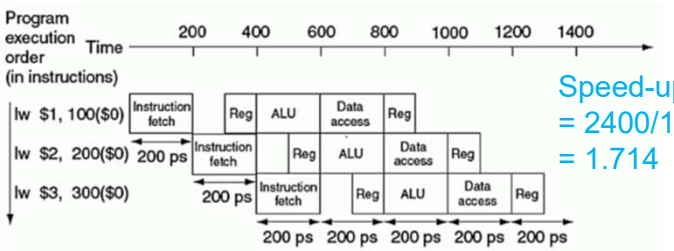
Instruction	IF	ID	EX	MEM	WB	Total
lw	200 ps	100 ps	200 ps	200 ps	100 ps	800 ps



CPU-time needed by a single-cycle processor = 2400 *ps*

Pipelining: Executing a Program Faster

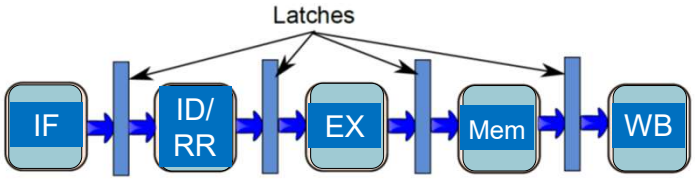
Instruction	IF	ID	EX	MEM	WB	Total
lw	200 ps	100 ps	200 ps	200 ps	100 ps	800 ps



Speed-up = 2400/1400 = 1.714

CPU-time needed by a 5-stage pipeline processor = 1400 *ps*

MIPS Pipeline: Why do you need inter-stage latches (Pipeline Registers)?

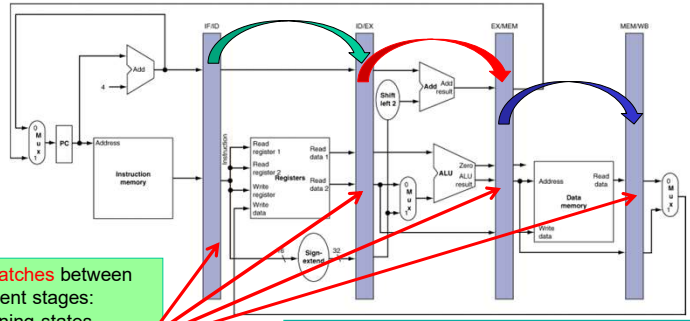


- Pipeline registers (latches) are inserted between two consecutive pipeline stages
- All the registers are synchronized with the system clock and seamlessly transfer instruction-bits and other intermediate data forward, across consecutive pipeline stages

Pipeline registers

- Need registers between stages
  - to hold information concerning the previous cycles
  - to pass information forward in every clock cycle

IF → ID/RR → EX → Mem → WB



Use latches between adjacent stages: pipelining states (space/time overhead)

Latch-contents should be propagated forward as the pipeline advances in sync. with clock cycles

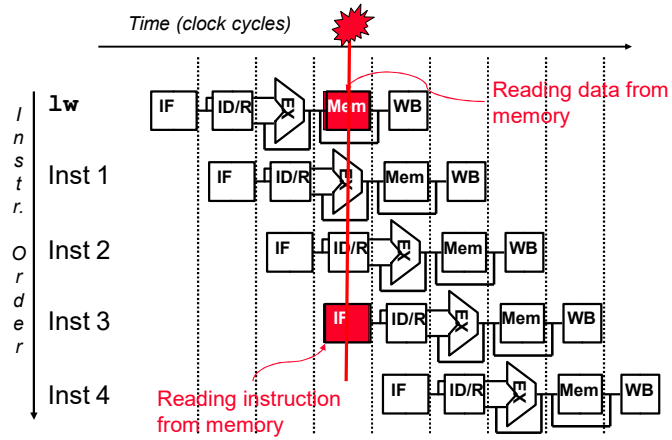
Pipeline Hazards

- Situations that prevent starting the next instruction in the next cycle; slows down the pipeline!
- Structural hazards
  - A required resource is busy
- Data hazard
  - Need to wait for previous instruction to complete its data read/write
- Control hazard
  - Deciding on control action depends on previous instruction

Hazards - Elaborated

- structural hazards: attempt to use the same resource by two different instructions at the same time
- data hazards: attempt to use data before it is ready
  - An instruction's source operand(s) are yet to be produced by a prior instruction, which is still in the pipeline
- control hazards: attempt to make a decision about program control flow before the condition has been evaluated and the new PC target address calculated
  - branch instructions
- Can always resolve hazards by waiting (inserting "bubbles" in the pipeline) – pipeline stall
  - pipeline controller must detect the hazard
  - and take action to resolve hazards
  - Bubbles may be needed to handle cache misses as well (may not be truly termed as hazard)!

Structural Hazard due to Memory Access Conflict



- *Solution*: Use separate instruction and data memory modules (Recall single-cycle case as well)

Structural Hazards

- Conflict for use of a resource
- In MIPS pipeline, with a single memory
  - Load/store requires data access
  - Instruction fetch would have to *stall* for that cycle
    - Would cause a pipeline "bubble" (stalls)
- Hence, pipelined datapaths require separate instruction/data memories
  - Or, separate instruction/data caches

Data Hazards

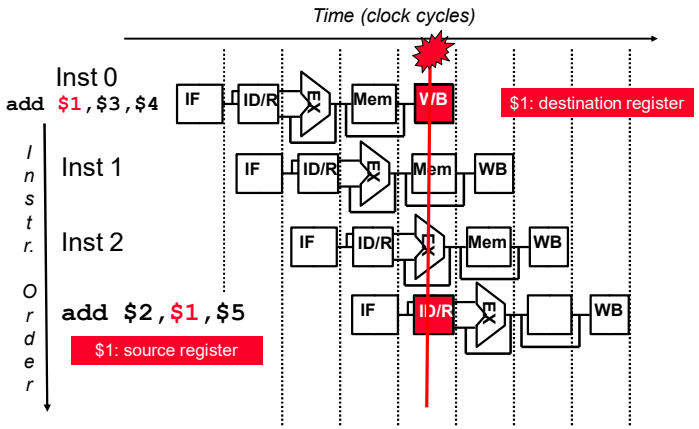
These occur when at any time, there are active instructions that need to access the same data (memory or register) locations

Where there's real trouble is when we have:

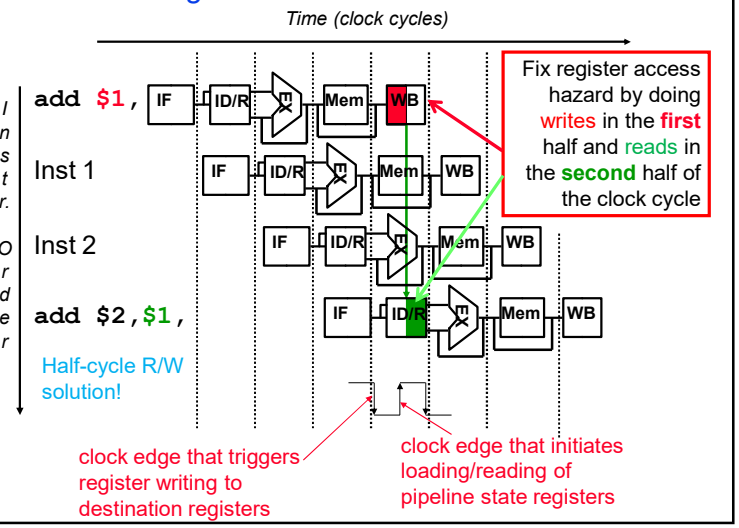
instruction A
instruction B

and B wants to manipulate (read or write) data before A does. This violates the order of the instructions, since the architecture implies that A completes entirely before B is executed.

Example: Hazards due to Register File Access

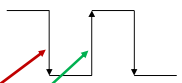


Solution: Register Write-Read Hazard?



Example: Half-Cycle Register Read/Write
(to mitigate data hazard)

```
add $1,$3,$4
lw …
sw …
add $2,$1,$5
```



Hardware solution:
Perform register writes in the **first** half of the clock cycle;
Then a read for the same register (holding the updated data) for a subsequent instruction, can be made in the **second** half of the same clock cycle;

Important Announcement

COS: Upcoming Classes

There will be one make-up class in the ONLINE MODE as mentioned below (Please mark your calendar);

1. Saturday, 04 April 2025; 10:10 AM – 11:40 AM (Make-up class)